

PROCESO DE DESARROLLO DE SOFTWARE

PROF. PEPE, JONATHAN

TRABAJO INTEGRADOR

Soko-Ban

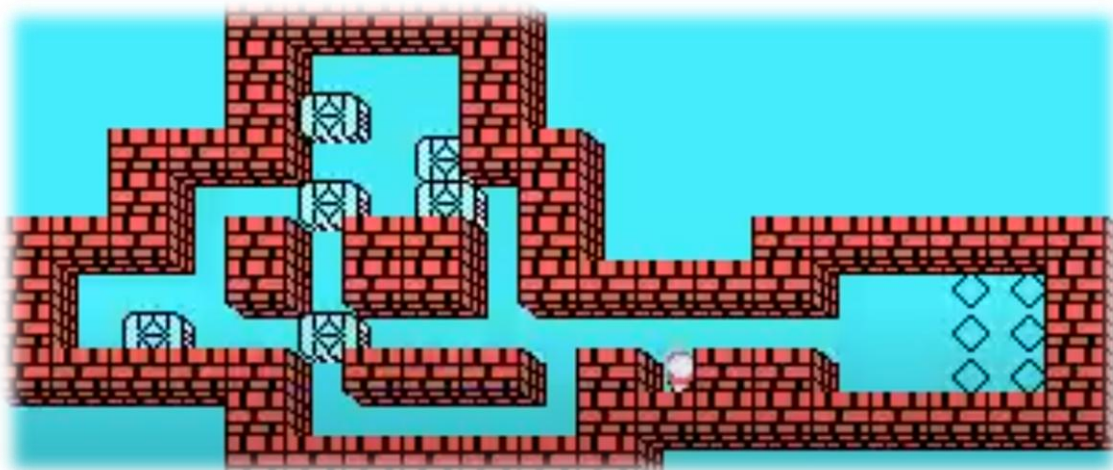
OBJETIVO

El objetivo de este trabajo es que el equipo de desarrollo aplique los **principios y patrones de diseño** presentados en la asignatura construyendo una versión del clásico videojuego *SOKOBan*.

DESCRIPCIÓN DEL JUEGO

El *SOKOBan* (*almacenista*) es un juego de lógica y rompecabezas *japonés* creado por *Hiroyuki Imabayashi* en 1980. El juego consiste en empujar *cajas* hasta su lugar correcto dentro de un *almacén* completando cada nivel de la manera más eficiente posible. La simplicidad y la elegancia de las reglas han hecho de *SOKOBan* uno de los juegos de ingenio más populares a nivel mundial.

GAMEPLAY



REQUISITOS FUNCIONALES

TABLERO DE JUEGO

El *tablero* se compone de los siguientes elementos:

- 🧩 *Paredes y espacios vacíos*
- 🧩 *Cajas normales, cajas frágiles y cajas llave*
- 🧩 *Casillas de destino, terrenos resbaladizos y casilleros cerrojo*
- 🧩 *Muros abiertos/cerrados*
- 🧩 *Jugador (sokoban)*

NIVELES

Los niveles se cargan desde diferentes *archivos de texto* (.txt) que contienen la disposición inicial del *tablero* en *caracteres*.

MOVIMIENTO DEL JUGADOR

El jugador se mueve con las *teclas direccionales* (*flechas*) y solo puede empujar las *cajas*, no tirar de ellas. Tampoco puede empujar más de una *caja* a la vez ni pasar a través de las *cajas*, *paredes* o *muros cerrados*.

TIPOS DE CAJA

- 🧩 *Normal*: Si el jugador intenta moverse a una casilla ocupada por una *caja normal* debe ser capaz de *empujarla* en su misma dirección. El movimiento de la caja en un *terreno resbaladizo* provoca que la misma se deslice hasta chocar con otro objeto o llegar a un *espacio vacío*. Las cajas no pueden atravesar otras *cajas*, *paredes* o *muros cerrados*.
- 🧩 *Frágil*: Conserva las mismas reglas de movimiento que las *cajas normales*, pero solo puede ser empujada una cantidad limitada de veces. Cada vez que el jugador la empuja, se reduce su *resistencia*. Cuando su resistencia llega a cero, la caja *se rompe*.
- 🧩 *Llave*: Conserva las mismas reglas de movimiento que las *cajas normales*, pero al ser colocada sobre un *casillero cerrojo*, abre los *muros* correspondientes.

DESHACER MOVIMIENTOS

Se permite deshacer los últimos *15 movimientos* realizados por el jugador. Para ello, debe existir un *botón undo* que, cada vez que se presione, restaure el estado del nivel *5 movimientos* hacia atrás, hasta un máximo de tres usos consecutivos.

CONDICIÓN DE VICTORIA

Se logra la *victoria* cuando todas las cajas están en sus destinos, avanzando al *siguiente nivel*.

PUNTAJE

Al finalizar cada nivel, se debe mostrar un resumen del desempeño del jugador, incluyendo *movimientos* realizados, *empujes* efectuados, uso del *botón undo* y, finalmente, el *puntaje final obtenido*. El criterio utilizado para calcular dicho puntaje debe ser definido por el equipo de desarrollo.

INTERFAZ DE USUARIO

En el *HUD* se debe mostrar la cantidad de *movimientos*, *empujes* y *nivel actual*. También deben existir botones visibles que permitan *deshacer movimientos* y *reiniciar el nivel actual*.

ELEMENTOS GRÁFICOS Y SONOROS

Todos los elementos del tablero deben ser representados por *imágenes* y las acciones del juego deben estar acompañadas de *efectos de sonido* apropiados. Todos los *assets* utilizados deben ser de dominio público.

FUNCIONALIDADES ADICIONALES

Cada equipo debe, necesariamente, incorporar **dos funcionalidades adicionales** al juego que impliquen la aplicación de patrones de diseño.

REQUISITOS TÉCNICOS

El proyecto debe ser desarrollado en *Java* utilizando *Swing* para la implementación de la *interfaz gráfica de usuario (2D)*.

ENTREGABLES

- 🚦 Link al repositorio público de *GitHub* con el código fuente del proyecto
 - Incluir archivo *README* con las instrucciones para ejecutar el juego
- 🚦 Video *gameplay* que muestre todas las funcionalidades implementadas
- 🚦 Diagrama de clases en *StarUML*
- 🚦 Informe técnico con las decisiones de diseño y su justificación

El trabajo debe entregarse vía correo institucional a la casilla del docente (jpepe@uade.edu.ar). El envío debe ser realizado por un solo integrante del grupo, con copia al resto de los miembros, indicando el nombre del *equipo de desarrollo* y el apellido y nombre de cada desarrollador/a.

CRITERIOS DE EVALUACIÓN

El foco de la evaluación es la **correcta aplicación de los principios y patrones de diseño** presentados en la asignatura (*SOLID, GRASP, MVC, GOF*), pero también se considera la creatividad, calidad del código, documentación y diseño general de la solución.

Cada integrante del equipo deberá realizar una defensa exhaustiva del proyecto (*oral y/o escrita*), lo que determinará la calificación final del mismo.

USO DE HERRAMIENTAS DE INTELIGENCIA ARTIFICIAL

El uso de herramientas de *inteligencia artificial* está permitido como **apoyo** al desarrollo, siempre que sea declarado en el informe técnico. El equipo es responsable de comprender, validar, adaptar y defender todo el código entregado, así como las decisiones de diseño adoptadas. La falta de comprensión del proyecto durante la defensa afectará significativamente la calificación final.

Good Luck!